

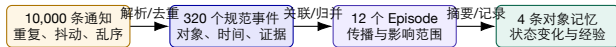
面向 AI 原生系统的智能运维

第 16 讲 | 告警事件分析与运维对象历史记录

从瞬时通知到可追溯、可复用、可更新的对象级事件记忆

陈鹏飞 | 中山大学
校企联合研究生课程 2026 秋

开场：一万条告警为什么不能直接写进“记忆”？



- ▶ 原始告警描述“规则在某个时刻满足”，并不自动代表一次独立故障；
- ▶ 事件系统要保留原始证据，同时生成稳定对象、生命周期和关系；
- ▶ 长期记忆只保存对未来判断有用、可追溯且仍然有效的知识。

本讲核心 事件处理不是压缩文本，而是把通知流转化为有身份、有时间、有证据、有状态的知识对象。

去重算例：三条通知应生成几个事件实例？

课程规则：指纹由规则、对象 UID 和区域构成；状态与观测值不进入指纹，恢复消息更新同一实例。

到达顺序	内容	指纹 / 实例	动作
1	u1; HighLatency; FIRING	f1 / e1	创建 e1
2	u1; HighLatency; FIRING	f1 / e1	更新 last-seen 与计数
3	u1; HighLatency; RESOLVED	f1 / e1	关闭 e1
4	u2; HighLatency; FIRING	f2 / e2	创建另一实例 e2

此规则下前三条是一个告警实例的生命周期；不同 UID 不能因名称相同而合并。

推导与判断 相同指纹在恢复后再次触发时，应按明确的重开策略创建新 Episode 或关联旧 Episode，避免把数月历史压成一条“长期故障”。

算例使用假设数据。

边界：第 15、16、17 讲分别交付什么？

章节	核心问题	主要产物	不在本讲重复
第 15 讲	世界里有哪些对象？	canonical object、关系、血缘、来源	不再展开实体对齐与对象抽取
第 16 讲	这些对象发生了什么？	event、episode、history、memory	不把相关事件直接宣布为根因
第 17 讲	为什么发生、下一步做什么？	假设、主动查询、证据排序、建议	不提前展开 Agent Loop 与修复执行

判定标准 第 16 讲的成功不是“已经诊断根因”，而是让第 17 讲能查询到高质量、低噪声、带时间和证据的历史上下文。

学习目标：学生最后应能独立设计一条事件管道

1. 区分 observation、alert、event、episode、incident、change 和 memory；
2. 设计兼容多源数据的 Canonical Event Schema 与状态机；
3. 掌握 fingerprint、时间窗、PMI、事件图、拓扑和语义推理等归并方法；
4. 读懂 COLA、iPACK、ESRO、AlertGuardian 的方法、结果和失败边界；
5. 设计证据约束的事件摘要，避免 LLM 把相关性改写成因果事实；
6. 将 Episode 更新为对象历史账本，并处理 TTL、冲突、过期和 supersedes；
7. 把 NSDI、OSDI、SIGCOMM 的深层观测信号翻译为可保存的事件字段。

2 学时主线：先规范事实，再组织记忆



- ▶ 前半段以字段、状态机、公式和小算例讲清事件管道；
- ▶ 中段用论文说明“统计过滤 + 知识推理 + 图关联”怎样组合；
- ▶ 后半段把 Episode 写入历史记忆，并用 AI 集群案例说明深层信号来源。

一、事件语义：究竟发生了什么？

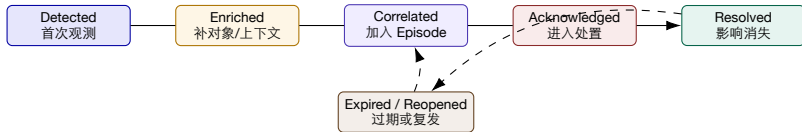
先统一 observation、alert、event、episode、incident 和 memory 的边界

六个概念：粒度不同，责任也不同

概念	典型来源	含义	示例
Observation Alert	metric/log/span/profile rule/monitor	一次原始观测 规则进入 Pending/Firing/Resolved	GPU memory 91% GPUMemoryHigh
Event Episode Incident Memory	事件管道 关联引擎 人/流程 历史账本	带对象、时间、证据的状态变化 同一传播过程中的事件集合 需要协同处置的业务影响 对未来仍有用的已验证经验	gpu-3 memory pressure model-v3 推理退化 Chat API P95 超 SLO 此版本在长提示下易 OOM

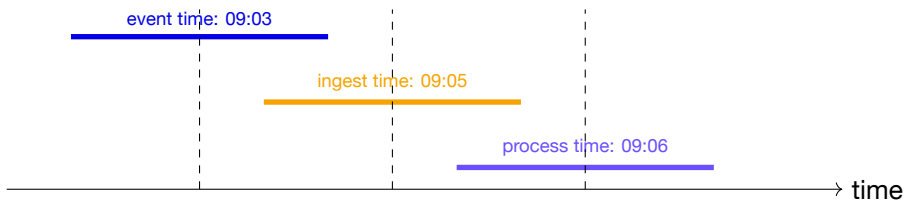
一条 Observation 可以不触发 Alert；多个 Alert 可归成一个 Event；多个 Event 可形成一个 Episode；是否升级为 Incident 取决于影响和治理规则。

事件生命周期：不是一条不可变记录



- ▶ 每次状态变化采用 **append-only transition**，不能覆盖原始事实；
- ▶ Reopened 应保留前一 Episode 的链接，便于判断复发而非新故障；
- ▶ Expired 表示证据不再新鲜，不等于事件从历史中删除。

三个时钟：事件时间、到达时间、处理时间



- ▶ 日志缓冲、Agent 离线、网络分区都会造成晚到事件；
- ▶ 关联窗口应主要基于 event time，并用 watermark 决定何时封闭 Episode；
- ▶ 审计还要保存 observed/ingest/process time，解释“系统当时知道什么”。

Canonical Event Schema: 最小字段也要支持审计

字段组	核心字段	作用
身份	event_id、source_event_id、schema_version	幂等、回放、演化
对象语义	object_id、object_type、scope event_type、symptom、severity、impact	与第 15 讲对象图连接 表达发生了什么
时间	start、last_seen、end、ingest、watermark	乱序、持续性、封闭
归并	fingerprint、dedup_key、correlation_keys	去重和关联
状态证据	lifecycle、count、ack、resolution evidence_refs、provenance、confidence	状态机和运营流程 防止摘要脱离原始数据
治理	owner、retention、privacy、super-sedes	访问、过期、冲突更新

设计原则 Schema 的目标不是“容纳所有字段”，而是让任意结论都能回到对象、时间和原始证据。

CloudEvents、Kubernetes Event、OTel Event 如何对齐？

语义	CloudEvents	Kubernetes Event	OpenTelemetry Logs/Events
身份	id、source	metadata.name/uid	TraceId/SpanId + event.name
类型	type	reason、action	event.name、severity
对象	subject	regarding、related	Resource / attributes
时间	time	eventTime、series.lastObservedTime	Timestamp、ObservedTimestamp
重复内容	扩展字段 data	series.count note	aggregation 由后端实现 Body + Attributes

- ▶ CloudEvents 适合作为跨系统信封；Kubernetes Event 强调对象状态变化；
- ▶ OTel Event 是有名称的日志记录，可与 Trace/Resource 连接；
- ▶ Canonical Schema 可以映射三者，但不能丢掉原始字段和规范版本。

证据、推断、处置结论必须分层

Evidence

- ▶ 指标点、日志行、Span、Profile；
- ▶ 采集时间、URI、哈希；
- ▶ 不带根因判断。

Inference

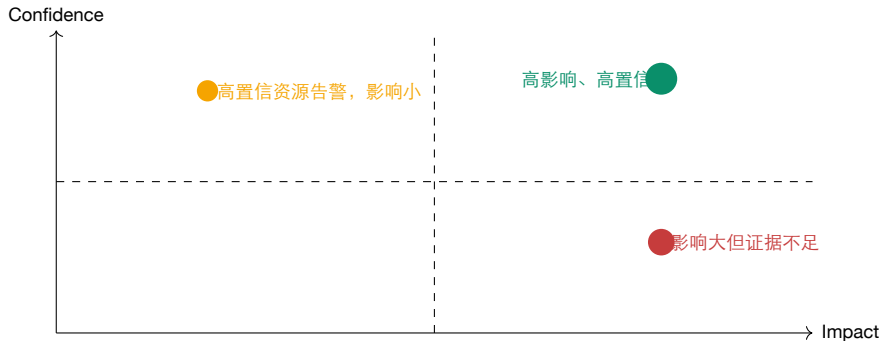
- ▶ “可能同一 Episode”；
- ▶ 关联分数与依据；
- ▶ 可被新证据推翻。

Decision

- ▶ 升级 Incident、指派、关闭；
- ▶ 由流程和权限决定；
- ▶ 必须记录决策人/规则。

常见错误 把“GPU 利用率下降”和“刚发生发布”直接摘要成“发布导致 GPU 故障”；前两者是事实，因果关系仍是待验证假设。

Severity、Impact、Confidence 是三个轴



- ▶ Severity 是通知/响应策略；Impact 是用户、SLO、任务和资源后果；
- ▶ Confidence 是解析或关联判断的证据强度；
- ▶ 高影响低置信事件应优先补证据，而不是被低置信度自动压制。

AI Native 事件分类：传统主机事件远远不够

层次	事件类型	代表字段
模型/数据 推理运行时 训练运行时 GPU/网络 RAG/Agent 流程/变更	model version、eval regression、drift、data gap queue、TTFT、TPOT、KV cache、batch、OOM rank、step、collective、checkpoint、straggler ECC、HBM、PCIe、NCCL、RoCE、path index、retrieval、tool、loop、memory、policy deploy、config、prompt、policy、rollback	model_digest、dataset_run、metric_delta request_class、engine、cache_state job、rank、step、collective、retry device、link、RNIC、ToR、flow trace、tool、memory_version、agent_run change_id、artifact_digest、actor

事件类型必须与对象图共同演化：模型、Prompt、Index、Agent 和 GPU 都要有稳定 object ID。

训练事件：一个“任务失败”至少要拆成四层



- ▶ Job event 表示业务任务状态；Step/Rank event 表示并行执行上下文；
- ▶ Collective event 描述通信阶段；Device/Path event 保存底层证据；
- ▶ 若只写“训练失败”，历史记忆无法区分数据错误、坏机器、网络、straggler 或软件异常。

推理事件：同一个超时可能来自完全不同阶段

Admission 限流、队列等待、模型未加载。

Prefill 长提示、算力饱和、KV 分配。

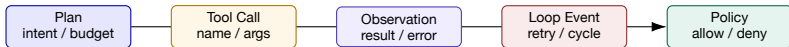
Decode batch 抖动、cache miss、抢占。

Tool/RAG 检索慢、工具超时、外部依赖。

事件示例

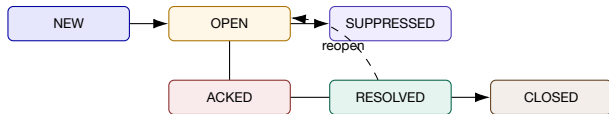
event_type: inference.latency_regression; object_id: endpoint/chat-prod; stage: prefill; symptom: TTFT p95 1.2s → 4.8s; model_version: v3; evidence: trace-9f, profile-18。

Agent 事件：要观察模型调用，也要观察控制循环



- ▶ 工具 500 错误、参数 schema 失败、预算耗尽、循环重复、策略拒绝都是事件；
- ▶ memory read/write 要记录 version、scope 和 conflict，避免多个 Agent 固化错误结论；
- ▶ Agent trace 是事件关联的重要 causation chain，但不等于底层系统因果链。

状态机细节：Firing 与 Resolved 之间还有什么？



- ▶ **SUPPRESSED** 只是通知策略，不应删除事件或证据；
- ▶ **RESOLVED** 表示观测恢复，**CLOSED** 表示流程和复盘完成；
- ▶ 自动关闭需要终态 **Oracle**：指标恢复、对象状态正常、持续观察窗通过。

事件完整性检查：写入之前先做十个问题

1. object_id 是否存在且在事件时间有效?
2. event_type 是否属于当前 schema version?
3. $start \leq last_seen \leq end$ 是否成立?
4. fingerprint 是否稳定、是否误含随机字段?
5. evidence_refs 是否可访问、是否有哈希?
6. severity 与 impact 是否被混用?
7. count 增长是否对应重复实例，而非不同症状?
8. lifecycle transition 是否符合状态机?
9. confidence 是否标明来自规则、模型还是人工?
10. retention/privacy 是否允许进入长期记忆?

本节小结：事件是“带时间的对象事实”

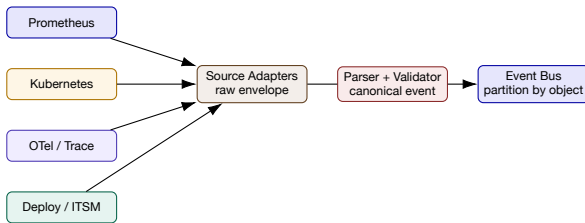
1. Alert 是规则状态，Event 是规范化状态变化，Episode 是关联集合；
2. Canonical Event 必须连接 object、time、evidence、lifecycle 和 governance；
3. 三个时钟、append-only transition 和 schema version 支持乱序与回放；
4. AI Native 事件要覆盖模型、训练、推理、GPU、网络、RAG 和 Agent；
5. 相关性、置信度、严重度和影响必须分开表达。

下一步 把 Prometheus、Kubernetes、日志、Trace 和变更记录转换成同一事件语言。

二、解析与规范化

从异构通知中提取对象、症状、时间、范围和证据

多源事件适配器：先保留原始信封，再映射语义



- ▶ raw envelope 原样保存，以便重放新 parser；
- ▶ adapter 只处理协议与认证，parser 负责语义；
- ▶ partition key 优先使用 canonical object，而不是来源系统名称。

原始示例：一条告警规则提供的信息其实很少

本地 LLM 部署课程中的 Prometheus 规则

```
alert = GPUHighMemory
expr = used_bytes / total_bytes > 0.9
for = 10m, severity = warning
summary = GPU memory usage high
description = GPU {gpu_id} memory at {value}%
```

- ▶ 它没有说明模型版本、请求类型、Pod、节点、部署变更和用户影响；
- ▶ **parser** 必须连接标签、对象图、Trace/Profile 与最近变更；
- ▶ 原规则只证明“阈值持续满足”，不能直接证明 OOM 或泄漏。

解析第 1 步：模板化，消除动态值噪声

原始文本

GPU 3 memory at 93.7% on pod
llm-server-7d9, node gpu-a12

事件模板

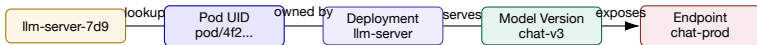
GPU <id> memory at <ratio> on pod
<pod>, node <node>

提取字段

- ▶ gpu_id = 3; ratio = 0.937;
- ▶ pod = llm-server-7d9;
- ▶ node = gpu-a12;
- ▶ template_id = gpu-memory-ratio-v2。

为什么先模板化 若把时间、数值、Pod 随机后缀全部放进 fingerprint，同一持续事件会被错误拆成无数新事件。

解析第 2 步：把字符串映射到对象图



- ▶ 事件主对象可选 Pod 或 Endpoint，但必须保存 scope 中的完整影响对象集合；
- ▶ object resolution 失败时标为 unresolved，不要让 LLM 猜一个相似名称；
- ▶ 对象在 event time 已删除时，查询历史版本而不是当前 CMDB。

解析第 3 步：补充语义，而不是补写根因

字段	合法补充	不应自动补充
symptom	memory pressure、ratio 0.937、持续 10m	memory leak
scope	pod、node、deployment、model、endpoint	全集群受影响
context	最近 30m 发布 model-v3	发布导致异常
evidence	metric range、profile URI、trace IDs	未采集到的 kernel 名
impact	TTFT p95 同窗上升 3.6s	用户全部失败
confidence	规则 1.0；对象映射 0.98；关联 0.72	“综合置信度 99%”

语义越权 Enrichment 的职责是补上下文；根因、责任和处置是后续推断/决策，不应在解析阶段被硬编码。

完整算例：从原始告警到 Canonical Event

字段	规范化结果
event_id	evt-20260822-gpu3-0007
object_id / scope	pod/4f2; node/gpu-a12; deployment/llm-server; model/chat-v3
event_type	resource.gpu.memory_pressure
time	start 09:03; last_seen 09:14; ingest 09:05
symptom	ratio 0.937, threshold 0.9, for 10m
fingerprint	hash(rule, pod_uid, gpu_uuid, severity)
context	change/model-v3 at 08:58; TTFT regression evt-09
evidence	prometheus://range/abc; profile://gpu3/18
confidence	parse 1.0; object 0.98; change-link 0.76

教学示例数字不代表真实生产数据；字段设计用于说明解析链路。

LLM 解析规则：只允许输出 schema，不允许自由发挥

输入

- ▶ 原始文本 + 来源类型；
- ▶ 可选 object candidates；
- ▶ event type 枚举；
- ▶ 时间和单位规则。

输出

- ▶ JSON schema 严格验证；
- ▶ 每个字段附 evidence span；
- ▶ unknown / ambiguous 显式返回；
- ▶ 不生成 root_cause 字段。

Verifier 规则

枚举校验、时间顺序、单位范围、对象存在性、证据 span 覆盖、敏感信息脱敏；失败则进入 deterministic fallback 或人工队列。

三层验证：语法正确还远远不够

Schema 字段类型、required、枚举、正则、范围。

Semantic 对象有效、单位合理、时间有序、事件类型匹配。

Evidence 每个非原始字段有来源；推断字段有方法和置信度。

- ▶ 例如 `gpu_memory=93.7` 语法正确，但单位可能是百分比而非 `bytes`；
- ▶ `object_id=pod/foo` 可能当前存在，但事件发生时尚未创建；
- ▶ `impact=all users` 若无 SLO/流量证据，应拒绝或降级为 `unknown`。

时间规范化：窗口选择会改变事件事实

- ▶ 统一时区并保存原始 offset，不能只存本地字符串；
- ▶ 解析 start、last_seen、end，避免只保留“收到通知的时间”；
- ▶ 对持续告警使用 interval，而不是把每个采样点当成独立事件；
- ▶ 对晚到数据设置 allowed lateness，例如 5 分钟 watermark；
- ▶ 对变更、Trace、Profile 使用各自 clock source，并保存 clock uncertainty；
- ▶ 对多集群事件使用 monotonic sequence 或 vector-like ordering 辅助判断。

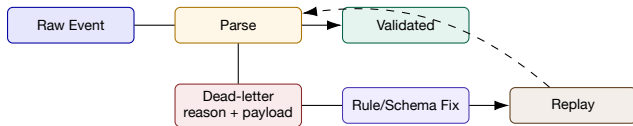
课堂反例 09:05 收到的日志记录可能描述 09:01 发生的异常；若按到达时间关联，它会被错误挂到 09:04 的发布之后。

Provenance: 每个字段都要回答“从哪里来”

字段	值	provenance
object_id	pod/4f2	Kubernetes UID lookup, rv=7812
model_version	chat-v3	Deployment annotation, observed 09:04
ratio	0.937	Prometheus sample, query hash 8a1
stage	prefill	Trace span attribute, span=9f32
related_change	change-27	temporal/topology linker v2, score 0.76
summary sentence 2	“TTFT 同窗上升”	event evt-09, evidence range 09:03–09:14

字段级 provenance 使未来可以用新 parser 或新规则重算，而不必相信旧摘要。

解析失败怎么办：保留失败记录并支持重放



- ▶ 失败原因分为 schema drift、unknown object、bad timestamp、privacy、parser bug；
- ▶ 监控 DLQ rate、oldest age、source distribution 和 replay success；
- ▶ 重放必须保持 source event ID，避免产生重复历史。

本节小结：解析的产物不是一段“更通顺的文本”

1. raw envelope、canonical event 和 derived fields 分层保存；
2. 模板化消除动态噪声，对象映射连接第 15 讲对象图；
3. LLM 只做受 schema、evidence span 和 verifier 约束的抽取；
4. 三个时钟、字段来源、失败队列和回放有助于排查问题；仍需测试数据演变和重放的一致性；
5. 下一步才讨论多个规范事件是否属于同一 Episode。

三、去重与归并

从告警风暴中恢复稳定事件身份和 Episode 边界

告警风暴：四种重复不能用一种规则处理

重复类型	例子	正确处理
同实例重复通知 对象副本重复 传播性重复 语义重复	同一 Alert 每 5 分钟重发 200 个 Pod 同时触发相同症状 DB 慢导致 API、队列、重试告警 “timeout” “deadline exceeded”	fingerprint 去重，更新 count/last_seen 合并为服务级 Episode，保留成员 建关联边，不把不同症状硬合成一条 模板/embedding 候选 + 证据验证

错误压缩 若把所有相近时间的告警直接合并，可能把同时发生的两个独立故障变成一个 Episode；若完全不合并，值班人员又会被通知淹没。

第一层：稳定 Fingerprint 解决 “同一个告警实例”

教学定义

$$f = H(\text{rule_id}, \text{object_id}, \text{stable labels}, \text{severity class})$$

- ▶ 放入稳定身份：rule、Pod UID/GPU UUID、cluster、tenant；
- ▶ 不放入动态值：当前数值、时间戳、随机 request ID、Pod 展示名后缀；
- ▶ schema/version 变化时维护 fingerprint version，避免新旧规则误合并；
- ▶ fingerprint 命中后更新 count、last_seen、min/max，而非覆盖 start。

Fingerprint 算例：哪些字段应该被排除？

字段	是否进入 fingerprint	理由
alertname=GPUMemoryHigh	是	规则语义
gpu_uuid=GPU-8a	是	稳定设备身份
pod_uid=4f2	是	区分工作负载实例
value=0.937	否	每个采样点变化
observed_at=09:07	否	每次通知变化
model_version=v3	视语义决定	若规则按模型版本独立处置则加入
request_id=abc	否	高基数，应作为 evidence member

结果 09:03–09:14 的 23 次通知被保留为一个事件：count=23, start=09:03, last_seen=09:14, max=0.962。

生命周期去重：Resolved 后多久算“新事件”？



- ▶ grace period 内再次触发：同一事件 Reopened，保留连续影响；
- ▶ 超过 cooldown：创建新 event，但用 recurrence_of 链接历史；
- ▶ grace/cooldown 应按事件类型设置：网络抖动与模型回归的时间尺度不同。

第二层：时间窗不是“越大越好”

固定窗 实现简单，但边界处相关事件会被拆开。

滑动窗 覆盖连续传播，但计算和重复候选增加。

Session 窗 按静默间隔封闭，适合突发 Episode。

- ▶ 候选窗只决定“哪些事件可以比较”，不是最终是否关联；
- ▶ 使用 event time + watermark，允许晚到数据补入尚未封闭的 Episode；
- ▶ 训练 collective 超时可用秒级窗，业务影响与工单可能需要小时级窗；
- ▶ 多尺度窗口可同时生成局部传播链和长期影响链。

第三层：用 **PMI** 衡量事件共现是否超出偶然

Point-wise Mutual Information

$$PMI(e_i, e_j) = \log \frac{p(e_i, e_j)}{p(e_i)p(e_j)}$$

- ▶ $PMI > 0$: 共同出现比独立假设更频繁;
- ▶ $PMI \approx 0$: 共现接近偶然; $PMI < 0$: 倾向不同时出现;
- ▶ 低频事件会产生不稳定高 PMI, 需要最小支持度、平滑和置信区间;
- ▶ PMI 只能表达历史关联, 不能自动解释方向与因果。

PMI 小算例：为什么“总是出现”的告警反而信息少？

100 个窗口中的计数

e_a 出现 40 次， e_b 出现 20 次，共同出现 16 次：

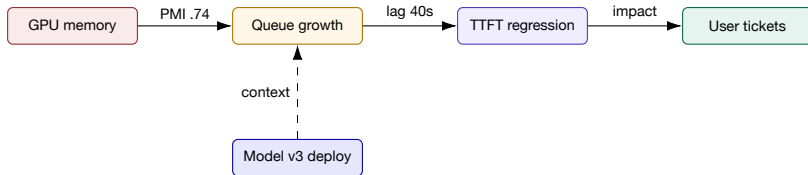
$$PMI(a, b) = \log \frac{0.16}{0.4 \times 0.2} = \log 2 > 0$$

- ▶ 若噪声事件 e_n 出现 95 次，和 e_b 共同出现 19 次：

$$PMI(n, b) = \log \frac{0.19}{0.95 \times 0.2} = \log 1 = 0$$

- ▶ 虽然共现次数 19 高于 16， e_n 几乎无判别力；
- ▶ 因此不能按“共同出现次数最多”直接构图。

从 pair score 到事件图：节点和边都要有语义



- ▶ 边保存 method、score、time lag、support、direction 和 evidence;
- ▶ cooccurred_with、precedes、affects、context_change 不可混成 related_to;
- ▶ 同一事件可加入多个候选 Episode，封闭时再做冲突消解。

噪声剪枝：高频背景事件会把图粘成一团

- ▶ 删除或降权在绝大多数窗口出现的 regular event；
- ▶ 过滤支持度过低的偶然边和高基数动态模板；
- ▶ 使用对象范围限制：不同 region/tenant 无共享路径时不比较；
- ▶ 对拓扑 hub 节点做 degree normalization，避免公共网关连接所有事件；
- ▶ 保留“被剪枝原因”，否则未来无法解释为什么某告警未进入 Episode；
- ▶ 对新事件保留探索预算，避免历史图永远看不到新型故障。

iPACK 的结果 论文的 GIP 仅保留约 2% 节点和 0.2% 边，同时将工单聚合 F1 从 0.743 提升到 0.884；数字限定于其 Azure 数据。

从边到簇：Connected Components 并不总是够用

方法	优点	风险
Connected Components	快、易解释	一条弱边可把两个故障错误连成一簇
DBSCAN	能识别噪声点	距离和密度参数对数据敏感
Community Detection	适合复杂图	稳定性、增量更新和解释较难
并查集 + 合并检查	在线高效	合并检查决定误关联是否扩散
Supervised pair-link	可融合多特征	需要标签，pair 正负极不平衡

工程常用 先用高精度规则/图边做 Union-Find，再对不确定边用模型或 LLM verifier；高影响 Episode 设置人工拆分/合并入口。

Merge 与 Link: 不是所有相关事件都应“融为一体”

Merge

- ▶ 同一事件实例的重复观测;
- ▶ 共享 event_id;
- ▶ 聚合 count、min/max、members;
- ▶ 可逆性较低, 需要严格门槛。

Link

- ▶ 不同症状但属于同一传播过程;
- ▶ 各自保留 event_id;
- ▶ 通过 episode_id 和 typed edge 连接;
- ▶ 允许后续拆分或调整关系。

关键区分 GPU memory、queue growth、TTFT regression 是三种事件, 通常应 Link; 同一 GPU memory 告警的 23 次通知才应 Merge。

归并评测：只看“告警减少率”会鼓励过度压制

Pair F1

相关事件对是否判断正确

B^3 / NMI

簇成员与真值簇一致性

压缩比

通知数 / Episode 数

Critical Recall

关键事件是否保留

- ▶ 再报告 detection-to-episode latency、人工拆分率、未知率；
- ▶ 评测真值来自工单/复盘/专家，会受组织流程影响；
- ▶ “压缩 99%” 可能意味着漏掉小范围但高风险事件；
- ▶ 生产验收要按严重程度、对象类型和新型事件分层。

离线评测陷阱：时间泄漏会让图模型看见未来

- ▶ 用全量历史构造 PMI/拓扑图，再在过去测试集上评估，会泄漏未来共现；
- ▶ 按随机事件对划分会让同一 Incident 同时出现在训练和测试；
- ▶ 使用最终工单标题作为在线可用特征，会虚高语义关联结果；
- ▶ 用已关闭事件的根因/处置字段生成摘要，不能代表事件进行时表现；
- ▶ 正确做法：按时间切分，训练图只使用当时可见数据，回放乱序和新服务；
- ▶ 还要分别评估已知模式、新对象、新事件类型和 schema drift。

COLA: 统计筛选与 LLM/SOP 推理的混合架构

Knowledge-aware Alert Aggregation in Large-scale
Cloud Systems: a Hybrid Approach

ICSE-SEIP '24, April 14–20, 2024, L

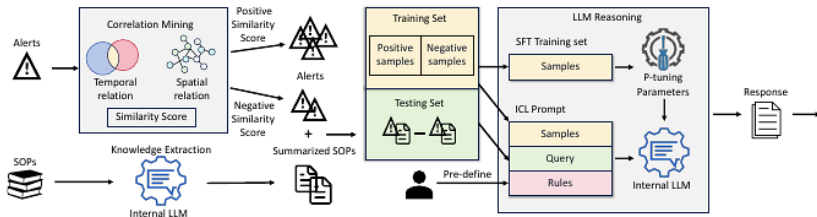


Figure 4: Overview of COLA

- ▶ 相关性挖掘先利用时间和服务拓扑快速处理大量候选；
- ▶ SOP 经过 LLM 摘要，提供影响、可能原因和处置知识；
- ▶ 只有不确定事件对进入 LLM reasoning，避免全量调用高成本模型。

COLA 第 1 层：时间关系 + 空间关系生成候选分数

时间关系

在滑动窗口中计算 $P(a_i|a_j)$ 与 $P(a_j|a_i)$ ，并用 Jaccard 类统计识别均匀出现的噪声事件。

空间关系

在服务拓扑图上做有方向的随机游走/序列采样，用 skip-gram/node2vec 风格表示学习得到服务距离。

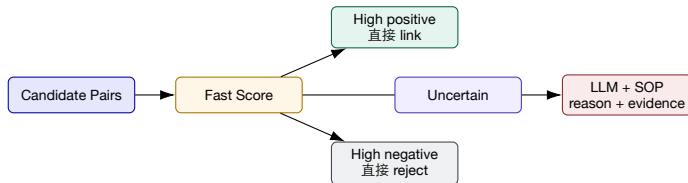
- ▶ 最终分数融合时间条件概率与空间 embedding 距离；
- ▶ 高置信正/负样本直接处理；不确定样本交给知识推理；
- ▶ 论文也指出历史拓扑不完整、新服务会削弱空间特征。

COLA 第 2 层：为什么要先摘要 SOP，再判断告警关系？

- ▶ 一个 SOP 可达 3–4 页，包含触发条件、影响、可能原因、处置步骤；
- ▶ 直接把两个长 SOP 放入 prompt 会超过上下文预算并稀释关键信息；
- ▶ 第一轮抽取结构化知识：impact / causes / mitigation / ownership；
- ▶ 第二轮输入两条告警、两份摘要、正负示例和输出规则；
- ▶ 输出不仅是 related/unrelated，还给出可读理由，供工程师审阅；
- ▶ 课堂迁移：企业 Runbook、模型卡、变更说明也可作为事件语义知识。

边界 SOP 可能过时或把经验假设写成事实，因此摘要必须保存文档版本和更新时间。

COLA 的分层路由：把 LLM 放在“高价值不确定区”



- ▶ 这是一种 cascade：便宜模型覆盖大多数，高成本模型处理决策边界；
- ▶ 需要校准两个阈值，并监控 uncertain rate；
- ▶ 当系统漂移时，uncertain rate 上升是比总 F1 更早的运营信号。

COLA 结果：高 F1 的同时，LLM 仍是主要延迟来源

0.901–0.930

三个生产数据集上的 F1

500k / 3k

约 50 万告警与 3000 份 SOP

4 months

论文报告的生产部署时间

- ▶ 完整 COLA 的 Precision/Recall 在三个数据集上分别形成 0.908、0.930、0.901 F1；
- ▶ 相比论文中复现的既有方法，F1 提升约 36.1%–47.1%；
- ▶ 平均推理时间约 5.78–8.94 秒/事件对，而纯 ICL 为 42.81–50.02 秒；
- ▶ 说明混合路由显著降低 LLM 成本，但仍不适合逐条原始通知全量调用。

COLA 消融：时间关系和 LLM 知识贡献最大

变体	Dataset A F1	Dataset B F1	Dataset C F1
去掉 temporal relation	0.550	0.551	0.563
去掉 spatial relation	0.884	0.856	0.863
去掉 LLM	0.616	0.613	0.638
完整 COLA	0.908	0.930	0.901

- ▶ 论文报告移除时间、空间和 LLM 后，平均 F1 相对下降约 39.3%、5.5%、31.8%；
- ▶ 空间贡献较小与拓扑数据不完整、新服务变化有关；
- ▶ 教学结论：知识推理不能替代高质量时间数据，拓扑也必须带 **freshness**。

AlertGuardian: 归并之后还要治理规则的完整生命周期

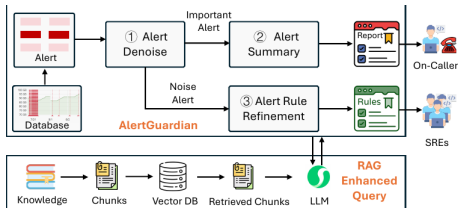


Fig. 9: Overview of AlertGuardian in Company-X.

- ▶ 在线轻量图模型处理高吞吐降噪；
- ▶ RAG/LLM 生成剩余关键告警摘要；
- ▶ 离线多智能体提出规则修改并回放验证；
- ▶ Reviewer + SRE 审批防止关键告警被误压制；
- ▶ 论文报告四个系统降噪
93.82%–95.50%， Action Accuracy
98.5%， RCA Accuracy 90.5%；
1174 条规则建议中 375 条被接受。

去重/归并方法对照：每层解决不同不确定性

方法	主要输入	擅长	失败边界
Fingerprint 窗口/PMI 拓扑/Trace Embedding LLM + Knowledge 人工反馈	稳定 labels/object 历史发生序列 对象图、调用链 标题/描述/SOP SOP/Runbook/上下文 工单/拆分/合并	同实例重复 高频共现 跨语义传播 语义近似 稀有、跨层关系 关键边界样本	schema/身份不稳定 新型/低频事件 图陈旧、Span 缺失 相似不等于同因 成本、幻觉、文档过时 成本与一致性

推荐架构 唯一键与原子更新控制重复写入，统计和图分析缩小候选，LLM 解释不确定关系，人工处理高影响冲突。

四、关联与影响范围

把多个事件组织为传播过程，但不把相关性冒充根因

事件关系 Schema: Episode 不是“一个大文本框”

关系	含义	需要的证据
duplicate_of	同一事件重复表示	fingerprint/identity
cooccurred_with	同窗显著共现	window、PMI、support
precedes	时间上在前	event time + uncertainty
propagates_to	沿依赖/调用路径传播	topology/trace + lag
affected	对用户/任务/SLO 产生影响	SLO、traffic、tickets
context_change	同窗存在变更	change object + time
recurrence_of	历史模式复发	pattern signature
contradicts	新证据与旧结论冲突	evidence comparison

边同样要保存 method、score、valid time、provenance 和 reviewer。

时间关联：先后顺序只是必要条件之一

一个可解释的时间分数

$$s_t(\mathbf{e}_i, \mathbf{e}_j) = \exp\left(-\frac{\max(0, t_j - t_i)}{\tau}\right) \cdot I(t_i \leq t_j + \epsilon)$$

- ▶ τ 表示该事件类型的传播时间尺度， ϵ 表示时钟不确定性；
- ▶ 网络 packet loss 到 collective timeout 可能是秒级；发布到缓存膨胀可能是分钟级；
- ▶ 双向同时出现的事件不能只靠时间确定传播方向；
- ▶ 时间分数应与拓扑、对象、语义和变更证据共同使用。

拓扑关联：路径存在不等于影响真的传播



- ▶ object graph 提供候选路径，Trace/flow/request evidence 证明本次确实经过该路径；
- ▶ 依赖边有方向、版本和权重；同一服务可能有 fallback 路径；
- ▶ 只沿静态拓扑扩散会扩大影响范围，必须结合当前流量与异常覆盖；
- ▶ 输出 “possible impacted” 与 “observed impacted” 两类集合。

变更关联：最近发布是上下文，不是默认根因

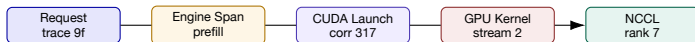
时间 变更是否在异常之前，并落在合理传播窗？

范围 变更对象是否覆盖异常对象和受影响流量？

对照 未变更实例、canary、其他 region 是否正常？

- ▶ 事件层只生成 context_change 边和 impact delta；
- ▶ 若 rollback 后恢复，可增加支持证据，但仍要记录同期负载等替代解释；
- ▶ 对模型/Prompt/Index/Policy 变更保存 artifact digest，而非只保存“发布成功”；
- ▶ 变更事件本身也进入对象历史，支持未来 recurrence 查询。

调用链关联：correlation ID 比“时间很近”可靠



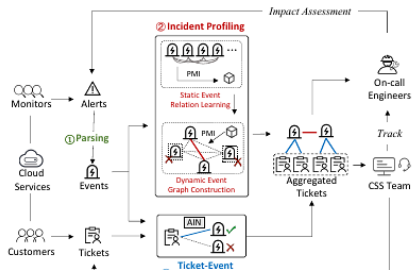
- ▶ Traceld/SpanId、parent range、thread、stream、device、correlation ID 共同重建链；
- ▶ 在高并发系统中，相邻时间的 kernel 可能属于另一请求；
- ▶ 事件图保存“本次执行”的 causation chain，而非仅保存服务拓扑；
- ▶ TELLER 等跨层方法说明，确定性对齐先于语言模型解释。

影响范围：从异常对象走到用户和任务

四层影响集合

1. **Direct objects**: 直接产生事件的 Pod/GPU/rank/endpoint;
 2. **Observed downstream**: Trace、请求、任务或工单实际覆盖;
 3. **Possible downstream**: 拓扑上可能受影响但尚无观测;
 4. **Business impact**: SLO、成功率、训练进度、成本、用户/租户。
- ▶ impact scope 随证据到达增量更新;
 - ▶ 摘要必须区分 observed 与 possible;
 - ▶ 用 region、tenant、model version、request class 做切片，避免“全量影响”夸大。

iPACK: 把系统告警和客户工单连接起来



- ▶ 告警先被解析为 event；GIP 用历史 PMI 与动态图构造 Incident profile；
- ▶ AIN 将文本、类别、对象和时间等特征组合，完成 ticket-to-event 链接；
- ▶ 即使工单描述语义不同，只要链接到同一事件图，也可归并为同一 Incident。

iPACK 的事件侧：静态关系学习 + 动态事件图

1. 使用 Drain 类日志模板解析，把告警文本转为 event template；
2. 按 monitor ID 与 owning component 分区，减少跨域噪声；
3. 在历史四小时滑窗上计算事件对 PMI，学习静态关系；
4. 运行时用最新窗口构图，以学习到的 PMI 连接候选；
5. 剪掉 regular/noisy events，留下 indicative events；
6. 每个连通子图形成 Incident profile，供工单链接。

教学重点 历史统计提供先验，动态图提供当前实例；两者分开才能处理“模式稳定但成员变化”的事件流。

iPACK 的工单侧：Attentive Interaction Network 学什么？

- ▶ Ticket 文本与 Event 文本分别使用 BERT 等表示；
- ▶ 类别特征包括产品、区域、时间、服务、严重度等；
- ▶ attentive feature interaction 学习“哪些字段组合”最能指向同一事件；
- ▶ 对每张工单输出 top-k event，随后通过 event-event graph 扩展到同一 Incident；
- ▶ 这比只做 ticket-ticket 文本相似更能处理级联影响造成的不同症状描述。

边界 若底层监控没有为受影响租户触发任何事件，工单无法及时链接；iPACK 的失败案例正暴露了这种 under-monitoring。

iPACK 结果：影响信息能显著提高重复工单归并

0.871–0.935

三个 Azure 数据集的 F1

12.4–31.2%

相对第二名方法提升

0.817

AIN 的 Acc@1

- ▶ AIN 的平均 top-k accuracy 为 0.887；注意力交互使平均准确率提升 17.8%；
- ▶ GIP 将事件图降到 2% 节点/0.2% 边，并将总 F1 从 0.743 提升到 0.884；
- ▶ 结果说明：客户影响和系统事件应该双向连接，而不是各自孤立归档；
- ▶ 数字来自论文三个 Azure 生产数据集，不能直接视为本课程案例性能。

iPACK 失败案例：没有事件，就无法被关联算法“猜出来”

论文案例

部分租户因 canary 配置错误出现 503，但监控按所有租户聚合，最初没有触发对应告警；约五小时后影响扩大，告警才出现并被 iPACK 连接到工单。

- ▶ 关联系统的上限由事件覆盖率决定；
- ▶ 租户/模型/版本等维度被聚合掉后，影响范围无法恢复；
- ▶ “未关联工单”应成为 observability gap 事件，推动新监控规则；
- ▶ 长期记忆不只记录故障原因，也要记录“当时缺了什么信号”。

Episode 装配：成员、主事件、影响和上下文分开

Episode Contract

episode_id: ep-20260822-17; members: evt-07, evt-09, evt-11; primary_symptom: TTFT regression; scope: model-v3 / chat-prod / 18% requests; context: change-27; impact: SLO burn 3.4x; status: open; uncertainty: root cause unresolved。

- ▶ primary symptom 用于展示，不应删除其他成员；
- ▶ change 是 context，不自动成为 cause；
- ▶ Episode 可以升级为 Incident，也可以在无业务影响时自动封闭为历史事件。

三类反例：关联分数高，仍然可能不属于同一故障

1. 共同上游负载：CPU 与延迟同时上升，实际都由促销流量导致；
2. 维护窗口共现：多个服务被计划性重启，事件高度同步但不是一个故障；
3. 公共 **Hub**：所有请求经过网关，拓扑距离近并不代表网关异常；
4. 周期性背景：每日备份与延迟总是同时出现，但今天的超时来自另一故障；
5. 标签污染：工单在复盘后被统一改写，造成离线语义相似度虚高。

反证字段 每个 Episode 保存 `alternative_explanations`、`missing_evidence` 和 `split_conditions`，供第 17 讲主动验证。

本节小结：关联的输出应是“可复查的 **Episode**”

1. 时间、拓扑、Trace、变更、语义和工单提供互补证据；
2. Merge 只用于同一事件实例，Link 用于传播链中的不同症状；
3. COLA 展示统计过滤 + LLM/SOP 推理；iPACK 展示事件图 + 客户影响；
4. 影响范围要区分 direct、observed downstream、possible downstream 与 business impact；
5. Episode 保留成员、关系、分数、反例和缺失证据，不提前宣判根因。

五、摘要与历史记忆

把 Episode 变成可引用的时间线、状态变化和处置经验

事件摘要不是“把所有文本压成一段话”

摘要要求

1. **What happened**: 对象、症状、开始/持续/恢复时间;
2. **What was impacted**: 观测到的范围与可能范围分开;
3. **What changed**: 相关变更与其证据强度;
4. **What is known/unknown**: 已证实事实、候选解释、缺失证据;
5. **What was done**: 动作、结果、验证窗、人工决策。

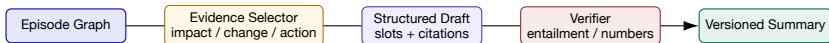
禁区 在根因尚未确认时，摘要只能写“与 model-v3 发布同窗、待验证”，不能写“model-v3 导致故障”。

四层摘要：为不同读者提供不同粒度

层次	内容	消费者
Headline	对象 + 主要症状 + 影响	值班列表、通知
Situation	时间线、范围、状态、关键证据	Incident 频道、负责人
Evidence Digest	成员事件、Trace/Profile/变更引用	诊断 Agent、专家
History Card	已验证原因、处置、结果、适用条件	对象记忆、复盘、流程生成

- ▶ 前三层可在事件进行时增量更新；History Card 通常在复盘/验证后写入；
- ▶ 同一摘要保留 version，每次更新记录新增/删除事实；
- ▶ 通知文本不是唯一真相，结构化事件图才是可重建来源。

Evidence-grounded Generation: 先选证据，再生成句子



- ▶ 每个句子带 supporting event/evidence IDs;
- ▶ 数字从结构字段填充，不由模型自由生成;
- ▶ Verifier 检查对象、时间、数值、状态和因果措辞;
- ▶ 无证据字段返回 unknown，必要时生成下一步采集问题。

摘要算例：同一 **Episode** 的“好版本”和“坏版本”

坏摘要 model-v3 发布导致 GPU 内存泄漏，引发所有用户请求超时，回滚后问题解决。

- ▶ “导致/泄漏/所有用户”无证据；
- ▶ 未说明时间窗和影响比例；
- ▶ 未引用成员事件。

好摘要 09:03–09:14, chat-prod 的 18% 请求 TTFT p95 从 1.2s 升至 4.8s；同窗 pod/4f2 的 GPU memory ratio 达 0.962。model-v3 于 08:58 发布，与事件存在时间/对象关联；根因尚未确认。

- ▶ 数字来自 evt-07/09；
- ▶ 明确 observed scope；
- ▶ 区分 context 与 root cause。

Oasis: 先评估 outage 影响范围，再生成可读摘要

- ▶ FSE 2023 的 Oasis 先聚合与同一 outage 相关的 incidents，评估 impact scope；
- ▶ 再使用微调的大语言模型生成面向值班人员的摘要；
- ▶ 影响评估组件在 Microsoft 使用超过三年，摘要组件在 18 个真实云系统上评估，并进行 outage owner 人工评价；
- ▶ 论文最终推动原型进入部分 Incident 团队的实验采用；
- ▶ 教学启示：范围评估应先于语言生成，摘要质量依赖上游聚合正确性。

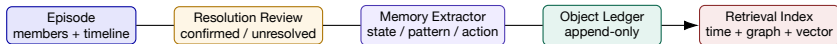
证据边界 论文摘要未提供可通用迁移的单一分数；课堂不虚构数字，重点采用其“先聚合/评估、后生成”的系统分层。

摘要评测：ROUGE 高不代表事实可靠

维度	指标/方法	典型问题
事实一致性	entailment、claim verification、数字对齐	是否编造因果/范围
覆盖度	key-field recall、timeline coverage	是否漏掉恢复与未知项
可读性	人工 Likert、长度、术语解释	是否便于值班理解
可操作性	next-step acceptance、action coverage	是否给出可验证下一步
引用质量	citation precision/recall	句子能否回到证据
新鲜度	summary lag、version staleness	新事件到达后是否及时更新

推荐门槛 高风险事件先过事实/引用门槛，再优化流畅性；“更像人工总结”不能抵消事实错误。

从 Episode 到 Object History: 写入的不是整段聊天记录



- ▶ Working memory 保存当前调查；Episode memory 保存本次过程；
- ▶ Object history 只抽取稳定状态变化、已验证模式、有效处置和证据引用；
- ▶ 原始 Episode 不删除，未来可用新知识重新解释。

Object History Ledger: 一条记忆的最小 Schema

字段	示例
memory_id / object_id	mem-318 / model/chat-v3
memory_type	state_change / recurring_pattern / remediation / gap
valid_time / observed_time	2026-08-22 09:03-09:22 / recorded 11:10
statement	长提示流量下 GPU memory pressure 与 TTFT regression 同现
conditions	prompt tokens > 16k; batch > 24; engine 0.9.2
status	confirmed / tentative / superseded / invalid
evidence_refs	ep-17; profile-18; change-27; review-6
confidence / reviewer	0.86 / SRE review + replay
expiry / supersedes	2026-11-22 / mem-211

Memory 的四种内容：事实、模式、动作、缺口

State 版本、容量、owner、依赖发生过什么变化。

Pattern 哪些症状/条件经常共同出现。

Action 做了什么、在什么条件下有效/无效。

Gap 缺了哪些指标、标签、Trace 或监控维度。

- ▶ 不把一次未经验证的候选根因提升为 semantic memory;
- ▶ 失败动作同样值得记录，可避免未来重复试错;
- ▶ Gap memory 能驱动可观测性建设和规则优化;
- ▶ Memory 的检索结果必须带当时条件和适用版本。

TTL、Decay 与 Supersedes: 经验会过期

- ▶ immutable fact (artifact digest、历史发生时间) 通常不衰减;
- ▶ runtime association (某版本常见症状) 随版本、流量和环境变化衰减;
- ▶ runbook/action memory 在工具/API/权限变化后需要 expiry;
- ▶ 新记忆不覆盖旧记忆, 而用 supersedes 建版本链;
- ▶ 检索分数同时考虑语义相似、对象距离、时间衰减、证据质量和状态;
- ▶ 被 superseded 的记忆仍可用于历史解释, 但不应作为当前默认建议。

教学检索分数

$$score = w_s s_{semantic} + w_g s_{graph} + w_t e^{-\Delta t / \tau} + w_e q_{evidence} - w_c conflict$$

冲突记忆：不要用“最后写入者获胜”

冲突示例

mem-211: “回滚 model-v2 后恢复”；mem-318: “重启 engine 后恢复，模型版本未变”。

1. 对齐两个记忆的对象、时间、条件、影响范围和动作顺序；
2. 检查是否描述不同 tenant/region/request class；
3. 保留两条陈述并建立 `contradicts` 或 `applies_when`；
4. 用回放、`canary` 或新增证据提升/降低置信度；
5. 无法消解时标为 `contested`，检索时展示冲突而非静默选一个。

从事件历史计算趋势：不是简单统计“发生过几次”

Frequency

单位时间 Episode 数

复发率

模式签名重复率

Severity Drift

影响是否逐步加重

Action Yield

处置成功/副作用

- ▶ 按对象版本和负载归一化，例如每百万请求事件数；
- ▶ 区分“监控变敏感导致事件增多”和“真实可靠性下降”；
- ▶ 复发签名可由 event types、topology motif、time lags 和 conditions 构成；
- ▶ 趋势输出要保留置信区间和数据覆盖变化。

Memory Compaction: 摘要旧记忆，但不删除证据链

- ▶ 将大量相似 Episode 合并为模式卡：频率、典型条件、例外、最近一次；
- ▶ 保留 representative episodes 与所有成员索引；
- ▶ 当新版本上线时创建新的 pattern segment，不把不同版本混合；
- ▶ 定期重新验证高价值动作记忆，过期动作降级为 historical；
- ▶ 向量索引可重建，ledger 与 evidence references 才是事实来源；
- ▶ compaction job 本身要有版本、输入范围、输出 diff 和回滚能力。

隐私与权限：历史记忆可能比原始告警更敏感

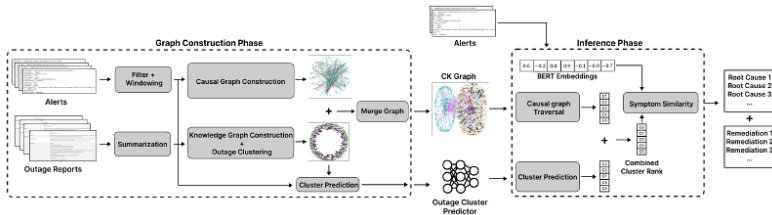
内容 Prompt、用户输入、工单、日志可能包含个人/业务数据。

推断 汇总多个事件可能暴露租户关系和内部拓扑。

动作 处置记录包含凭据位置、权限和安全控制。

- ▶ 写入前做字段分类、脱敏、tenant isolation 和 purpose binding；
- ▶ 检索按 object/tenant/role 过滤，不能“先召回再遮盖”；
- ▶ 摘要中的敏感 span 与原证据使用独立权限；
- ▶ retention、right-to-delete 与审计策略应覆盖向量索引和缓存副本。

ESRO: 把实时告警图与历史事故报告图合并



- ▶ 告警时间序列构造 causal graph；历史 outage report 提取 symptom、root cause、remediation，构造 knowledge graph；
- ▶ 通过事故时间窗把告警节点连接到历史症状节点，形成 CK graph；
- ▶ 推理阶段结合图遍历、症状相似与 outage cluster predictor，检索可能原因和处置。

ESRO 结果：实时结构与历史文本结合优于单一记忆源

+27%

根因推荐 ROUGE 平均提升

+39%

处置步骤 ROUGE 平均提升

62% / 72.7%

cluster predictor Top-1 / Top-5

- ▶ Cluster inference 的 root-cause ROUGE-1 为 0.242，两个基线为 0.207/0.176；
- ▶ remediation ROUGE-1 为 0.219，基线为 0.157/0.162；
- ▶ 数据来自一个大型 SaaS 企业约两年的事故，测试以约 50 个随机事故为主；
- ▶ 论文也观察到简单文本相似有时优于 GCN，说明复杂图模型不保证更好。

六、近年系统论文：哪些深层信号应升级为事件？

从 NSDI 2024-2026、OSDI 2025 和 SIGCOMM 2024-2025 提取事件字段

论文阅读地图：本节不讲“论文列表”，而讲事件传感器

层次	代表工作	深层信号	进入事件系统
训练任务 AI 基础设施	MegaScale、Minder、Aegis TPUv4、Astral	step/rank/collective/straggler link/power/cooling/log correlation	training.* event infra.* event
观测数据面 语义正确性 恢复安全	μ View、KPerfIR T2C PILOT	sketch、kernel internal events silent semantic violation action effect/cross-component side effect	profile/metric event semantic.* event recovery.* event
网络路径	R-Pingmesh、Crux、HPN	RNIC/path/contention/ToR	network.* event

NSDI 2024: MegaScale 与 TPUv4 把“稳定性”变成事件工程

MegaScale

- ▶ 深栈监控组件与事件；
- ▶ 定位 failure/straggler 并构造容错；
- ▶ 175B/12,288 GPU 达 55.2% MFU, 较 Megatron-LM 1.34 $\times$ 。

TPUv4 Resiliency

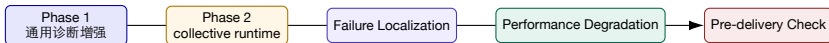
- ▶ 检测 machine/chip/link failure；
- ▶ 自动重配置 3D torus 路由并触发维护工作流；
- ▶ 报告 99.98% 可用性，约 1% 作业经历硬件 outage。

事件字段 job/step/rank、straggler type、failed link/chip、route reconfiguration、recovery outcome、maintenance correlation。

NSDI 2025: Minder/Holmes 说明 “任务失败前” 已有可记录的事件

- ▶ Minder 比较同一训练任务中的同伴机器，以逐指标去噪、持续性和优先序发现坏机器；
- ▶ 论文报告生产环境平均 3.6 秒反应、Precision 0.904、F1 0.893；
- ▶ Holmes 将问题提升为超大 GPU 集群中的 irregularity localization，强调训练并行角色与跨层上下文；
- ▶ 第 16 讲不重复检测算法，而把输出规范化为 `faulty_machine_candidate`、`metric_window`、`peer_group`、`persistence`；
- ▶ 若训练后来正常完成，也应将候选事件标为 `disproved/expired`，避免历史只保存 “命中”。

NSDI 2025 Aegis: collective library 本身成为事件传感器



- ▶ 无需修改客户代码，在 collective communication library 中定位运行时故障；
- ▶ 生产一年后，论文报告诊断 idle time 降低 >97%，任务 restart 降低 84%，性能退化降低 71%；
- ▶ 事件系统应记录 collective、rank、step、communication phase、fault scope 与是否需要 restart；
- ▶ 这些字段比“主机 CPU 高”更适合形成训练对象历史。

NSDI 2026: μ View 与 PILOT 把观测/恢复也变成事件

μ View

- ▶ IPU 上的轻量观测数据面；
- ▶ streaming sketch 细粒度处理指标；
- ▶ 提前暴露 SLO 风险并选择高信息信号；
- ▶ 事件字段：sketch error、sampling budget、risk signal。

PILOT

- ▶ 研究 75 个真实 recovery failure；
- ▶ 生产内 dry-run 恢复动作，观察跨组件副作用；
- ▶ 五个系统中发现 20 个恢复故障的 17 个；
- ▶ 事件字段：proposed action、simulated effect、side effect、oracle。

OSDI 2025: 静默语义违例和 GPU 内核内部事件

T2C Semantic Checkers

- ▶ 从测试代码静态/动态导出运行时 checker;
- ▶ 四个分布式系统生成数十到数百个 checker;
- ▶ 检出复现的 20 个真实 silent failure 中的 15 个;
- ▶ 事件类型: semantic.invariant_violation。

KPerfIR

- ▶ profiler 能力作为编译 pass 集成到 Triton;
- ▶ 观测 GPU kernel/thread-block/function-unit;
- ▶ 报告开销 8.2%，相对误差 2%;
- ▶ 事件类型: kernel.internal_stall / overlap gap。

SIGCOMM 2024: 网络事件必须说明“路径”和“服务影响”

- ▶ R-Pingmesh 用 commodity RNIC 端到端 probing 分离网络 RTT 与端主机处理延迟；
- ▶ 区分 RNIC 丢包和网络内丢包，并判断问题是否影响服务；
- ▶ 官方摘要报告数万 RNIC 部署超过 6 个月，一个月评估中 85% 定位准确，157 个交换机问题全部准确，覆盖 14 类问题；
- ▶ Crux 将 GPU intensity 与通信竞争关联，96-GPU 测试床 GPU utilization 提升 8.3%–14.8%；
- ▶ Alibaba HPN 的 dual-plane/dual-ToR 说明 path、ToR、hash polarization、link failure 都应成为可查询事件。

SIGCOMM 2025 Astral: 供电、散热、网络 and 日志进入同一事件空间

- ▶ Astral 面向超大规模 LLM 训练数据中心，采用 same-rail tier-2 network；
- ▶ 高密度部署引入分布式高压直流供电与风液混合冷却；
- ▶ full-stack monitoring 使用 cross-host hierarchical logging correlation，在规模化环境精确定位故障；
- ▶ Seer 生成 operator-granular execution timeline，服务诊断、调优和网络升级；
- ▶ 论文摘要报告基础设施逐步部署超过 18 个月，支持多客户训练与推理；
- ▶ 对象历史必须跨 compute/network/power/cooling，而非只保存软件告警。

把论文信号翻译为 Canonical Event 字段

工作	事件对象/类型	关键上下文
Minder/Aegis	machine/rank/collective fault	peer group、step、metric window、restart
MegaScale/TPUv4	straggler/link/chip/reconfig	job progress、route、recovery outcome
μView	SLO risk / sketch anomaly	sampling budget、resolution、forecast horizon
PILOT	recovery simulation result	action、predicted effect、side effect、oracle
T2C	semantic invariant violation	checker source、test provenance、affected state
KPerfIR	GPU intra-kernel event	kernel、block、unit、measurement error
R-Pingmesh/Astral	RNIC/path/power/cooling event	service impact、cross-host logs、timeline

统一接口 无论信号来自指标、runtime library、compiler profiler 还是网络探针，写入前都要连接 object、event time、evidence、scope 和 lifecycle。

七、贯穿案例与课堂实验

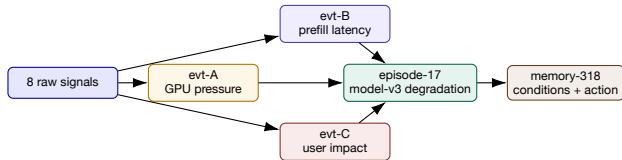
从 model-v3 发布到对象历史记忆，完整走一遍事件管道

贯穿案例：一个推理退化 Episode 的原始时间线

时间	原始信号	对象/症状	初始判断
08:58	Deploy Event	model-v3 / engine 0.9.2	context change
09:03	GPU alert	pod/4f2 gpu-3 / memory 93.7%	resource event
09:04	Profile	KV allocation +38%	evidence, not alert
09:05	Trace	prefill queue wait p95 +2.9s	latency event
09:06	SLO alert	chat-prod TTFT p95 4.8s	impact event
09:08	Tickets	长提示请求超时	observed user impact
09:16	Action	降低 max batch	mitigation attempt
09:22	Oracle	TTFT 恢复并持续 10m	resolved evidence

教学演示数据；用于说明转换链，不宣称真实生产结果。

案例转换：8 条信号如何变成 3 个 Event + 1 个 Episode?



- ▶ Deploy 保留为 context change，不与症状 merge；Profile/Trace 作为 evidence member；
- ▶ 三个事件由对象、时间、Trace 和影响连接到 episode-17；
- ▶ 只有“batch 调整后恢复且回放确认”的条件/动作进入长期记忆；
- ▶ 根因仍可标为 tentative：模型版本、batch 与长提示组合导致资源压力。

课堂实验：实现一个最小 Event Memory Pipeline

1. 读取本地 `alerts.yml`，定义 4 种 canonical event types；
2. 为 20 条模拟通知计算 fingerprint，输出 count/start/last_seen；
3. 在 5/15/60 分钟窗口计算共现与 PMI，说明最小支持度；
4. 接入对象图，生成 topology candidate edges；
5. 用规则或受约束 LLM 生成 Episode summary，并给每句 evidence IDs；
6. 将 confirmed state/action/gap 写入 object ledger；
7. 设计一个晚到事件、一个冲突记忆和一个未触发监控的工单反例；
8. 做时间切分评测，报告 pair F1、cluster metric、critical recall、summary factuality。

实验验收与研究问题

工程验收

- ▶ 幂等重放不产生重复历史；
- ▶ 晚到事件可更新未封闭 Episode；
- ▶ 摘要每句可回到证据；
- ▶ supersedes/contested 可查询；
- ▶ 不泄露跨租户敏感信息。

研究问题

- ▶ 新型低频事件如何归并？
- ▶ 图、文本、时间如何联合校准？
- ▶ 如何评估 memory 对未来 MTTR 的真实贡献？
- ▶ LLM 如何在 schema 演化下保持事实一致？
- ▶ 如何用 Agent 反馈更新记忆又避免错误固化？

乱序推演：恢复通知先到，如何避免把事件重新打开？

同一来源带单调递增修订号；处理端按事件时间与修订号更新状态，同时保留全部原始记录。

到达顺序	事件时间 / 修订号	内容	处理后状态
先到	10:05 / r2	RESOLVED	已恢复，等待补全历史
后到	10:00 / r1	FIRING	仍已恢复；补入开始时间
再到	10:00 / r1	重复 FIRING	不改变状态；增加接收记录
更新触发	10:08 / r3	FIRING	按重开策略创建新实例

若来源没有可比较修订号，需建立来源内排序规则；跨来源冲突应保留，不能强行比较各自序号。

推导与判断 按到达顺序覆盖状态，会把迟到的 r1 当成新故障。对象记忆须能修订历史摘要，并保存原始证据和变更原因。

算例使用假设数据。

参考论文与规范

1. Kuang et al., *Knowledge-aware Alert Aggregation in Large-scale Cloud Systems: a Hybrid Approach*, ICSE-SEIP 2024。
2. Jiang et al., *Incident-aware Duplicate Ticket Aggregation for Cloud Systems*, 2023。
3. Chakraborty et al., *ESRO: Experience Assisted Service Reliability against Outages*, 2023。
4. Jin et al., *Assess and Summarize*, ESEC/FSE 2023; Yu et al., *AlertGuardian*, ASE 2025。
5. *MegaScale*、*Resiliency at Scale*, NSDI 2024; *Minder*、*Evolution of Aegis*, NSDI 2025。
6. *μView*、*Pilot Execution*, NSDI 2026。
7. *Deriving Semantic Checkers from Tests*、*KPerfIR*, OSDI 2025。
8. *R-Pingmesh*、*Crux*、*Alibaba HPN*, SIGCOMM 2024; *Astral*, SIGCOMM 2025。
9. CloudEvents 1.0.2; Kubernetes Event v1; OpenTelemetry Logs/Events; Prometheus Alertmanager 官方文档。
10. 本地材料: ai-infra-engineer-learning-main 的 alerting、observability、GPU profiling 与 LLM deployment 练习。

论文结果均标明来源和适用条件; 外部图仅使用独立架构图裁剪, 不使用完整第三方 PPT 页面。